

Cahier des charges

Intitulé du projet

**Système de Détection d'Opportunités
d'Amélioration Continue des Processus
Bancaires via l'Intelligence Artificielle et la
RPA**

Réalisé par

Nader Khaskhoussy

Entreprise d'accueil



Siège Social Du Attijari Bank

Encadrant Entreprise

Firas El Abed

Encadrant SESAME

Wajih Ben Belgacem

Année Universitaire :

2025/2026

République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Cahier des Charges
Projet de Fin d'Études

Système de Détection d'Opportunités
d'Amélioration Continue des Processus
Bancaires via l'Intelligence Artificielle
et la RPA

Réalisé par :	Meriam
Organisme d'accueil :	Attijari bank Tunisie
Encadrant Entreprise :	Firas Elabed (Attijari bank)
Encadrant SESAME :	Wajih Ben Belgacem
Sujet :	Sujet 21
Spécialité :	Génie Logiciel
Durée prévue :	8 semaines
Équipe :	Rationalisation, Reengineering & Efficacité Opérationnelle

Année Universitaire 2025 – 2026

Table des matières

1	Contexte et Problématique	2
1.1	Contexte du Projet	2
1.2	Problématique	2
2	Objectifs du Projet	3
2.1	Objectifs Principaux	3
3	Acteurs et Cas d'Utilisation	4
3.1	Identification des Acteurs	4
3.2	Cas d'Utilisation Principaux	4
4	Besoins du Système	5
4.1	Besoins Fonctionnels	5
4.2	Besoins Non-Fonctionnels	5
5	Choix Technologiques et Architecture	6
5.1	Frontend (Interface Utilisateur)	6
5.2	Backend (API et Logique Métier)	6
5.3	Intelligence Artificielle et Data Science	6
5.4	Automatisation et Déploiement	7
6	Planification et Méthodologie	8

Chapitre 1

Contexte et Problématique

1.1 Contexte du Projet

Le secteur bancaire tunisien connaît une transformation numérique majeure. Dans ce cadre, **Attijari bank Tunisie** cherche continuellement à optimiser ses processus internes afin de garantir une haute qualité de service. Le département IT gère un afflux quotidien important de réclamations de la part des collaborateurs et des clients, allant des problèmes de réseau et d'accès VPN aux défaillances de sécurité et de systèmes de paiement.

1.2 Problématique

Actuellement, la gestion de ces incidents IT souffre de plusieurs limites :

- **Traitement manuel** : La catégorisation et la priorisation des tickets sont réalisées par des opérateurs humains, ce qui génère des retards.
- **Analyse réactive** : Les pannes sont traitées après leur signalement, sans aucune capacité d'anticipation ou de prédiction des incidents futurs.
- **Manque de capitalisation** : Bien que la banque dispose d'un vaste historique d'actions correctives, ces données ne sont pas exploitées pour suggérer automatiquement des résolutions rapides aux nouveaux problèmes similaires.

Ce projet de fin d'études a donc pour but de concevoir et développer une plateforme centralisée capable de comprendre automatiquement les réclamations, de prédire le risque d'incident global, et d'automatiser les actions correctives.

Chapitre 2

Objectifs du Projet

La solution proposée consiste en un **système intelligent et automatisé** agissant comme un assistant proactif pour les équipes opérationnelles IT.

2.1 Objectifs Principaux

1. **Analyse Sémantique (NLP)** : Implémenter un pipeline capable d'extraire le contexte, les entités et l'intention des réclamations textuelles.
2. **Prédiction d'Incidents (Deep Learning)** : Développer un réseau de neurones récurrents (LSTM) pour analyser les séquences temporelles des tickets et prédire la probabilité de pannes systémiques.
3. **Recommandation de Solutions (KNN)** : Créer un moteur de recommandation basé sur l'historique pour suggérer instantanément l'action corrective la plus appropriée avec un score de confiance.
4. **Automatisation Robotisée (RPA)** : Concevoir des robots logiciels (UiPath) capables d'exécuter automatiquement les actions correctives sur les serveurs pour les incidents les plus critiques.
5. **Tableau de Bord et Chatbot** : Offrir une interface Web (SPA) moderne permettant aux utilisateurs de soumettre des réclamations via un chatbot et aux responsables IT de superviser le flux via un dashboard dynamique.

Chapitre 3

Acteurs et Cas d'Utilisation

3.1 Identification des Acteurs

- **L'Utilisateur (Employé de la banque)** : Soumet des réclamations, interagit avec le chatbot, consulte l'état de ses tickets.
- **Le Responsable IT** : Supervise les anomalies via le tableau de bord, valide les recommandations de l'IA, déclenche les actions RPA.
- **L'Administrateur** : Gère les comptes utilisateurs, les rôles et consulte le journal d'audit (Audit Trail) pour des raisons de conformité.
- **Le Système Automatisé (Robot RPA)** : Acteur non-humain qui lit les alertes critiques et exécute les scripts de résolution sur le système d'information.

3.2 Cas d'Utilisation Principaux

- **S'authentifier** : Tout acteur doit s'authentifier de manière sécurisée (JWT).
- **Soumettre une réclamation** : L'utilisateur décrit son problème au chatbot.
- **Prédire un incident** : Le système analyse en arrière-plan le volume de tickets et génère une alerte prédictive.
- **Consulter le dashboard** : Le responsable IT visualise les graphiques, les scores d'anomalie et les alertes critiques.
- **Traiter une réclamation** : Le responsable IT valide ou modifie la solution recommandée par l'IA et déclenche son exécution.

Chapitre 4

Besoins du Système

4.1 Besoins Fonctionnels

- Le système doit fournir une interface conversationnelle (chatbot) persistante pour la capture des tickets.
- Le système doit nettoyer et vectoriser le texte des réclamations (spaCy/BERT).
- Le système doit calculer un "Score d'Anomalie" pour chaque nouveau ticket.
- Le système doit afficher des graphiques en temps réel (répartition par sévérité, évolution sur 7 jours).
- Le système doit intégrer un journal d'audit enregistrant chaque action effectuée, la date, et l'identité de l'utilisateur.
- Le système doit permettre au robot RPA de récupérer les alertes critiques via une API dédiée et de signaler leur résolution.

4.2 Besoins Non-Fonctionnels

- **Sécurité** : Utilisation stricte de JWT pour l'authentification. Les mots de passe doivent être hachés avec bcrypt.
- **Performance** : L'interface Web doit être une Single Page Application fluide, avec un rendu des graphiques sans surcharge du DOM.
- **Architecture** : La solution doit être conteneurisée pour faciliter son déploiement sur les serveurs de la banque.
- **Ergonomie** : L'interface doit respecter les couleurs et la charte graphique d'Attijari bank (Thème sombre/lumineux avec touches d'orange).

Chapitre 5

Choix Technologiques et Architecture

Afin de garantir la cohérence, les performances et l'évolutivité du système, la stack technologique suivante a été sélectionnée :

5.1 Frontend (Interface Utilisateur)

- **Angular 21 (TypeScript)** : Pour la création d'une interface SPA robuste, modulaire et hautement réactive.
- **HTML5/CSS3 Natif** : Conception d'un design system personnalisé (glassmorphism, variables CSS) sans recours excessif à des bibliothèques externes lourdes.

5.2 Backend (API et Logique Métier)

- **Python 3.11 avec FastAPI** : Choisi pour sa très haute performance, sa gestion asynchrone native et sa parfaite intégration avec l'écosystème IA et Data Science de Python.
- **PostgreSQL** : SGBD relationnel robuste pour le stockage transactionnel sécurisé des utilisateurs, des réclamations et de l'audit trail.
- **SQLAlchemy** : ORM (Object-Relational Mapping) facilitant l'interaction asynchrone avec la base de données.

5.3 Intelligence Artificielle et Data Science

- **spaCy & BERT (sentence-transformers)** : Pour l'analyse syntaxique et sémantique (NLP).
- **TensorFlow (Keras)** : Pour l'architecture, l'entraînement et le déploiement du réseau de neurones récurrents (LSTM).
- **Scikit-Learn** : Pour l'algorithme K-Nearest Neighbors (KNN) utilisé dans la recommandation des actions correctives.

5.4 Automatisation et Déploiement

- **UiPath Studio** : Pour la modélisation des processus robotisés (RPA) interagissant avec les API REST du système.
- **Docker & Docker Compose** : Pour orchestrer l'infrastructure (PostgreSQL, Redis, Elasticsearch, MLflow) et garantir l'isolation des services.

Chapitre 6

Planification et Méthodologie

Le projet sera mené selon une approche **Agile itérative**, adaptée aux contraintes temporelles d'un PFE de 8 semaines, réparti en 4 phases principales :

TABLE 6.1 – Macro-planning du Projet

Phase	Période	Livrables Attendus	Jalon
Phase 1	Semaine 1–2	Conception de l'architecture, développement de la BDD PostgreSQL et des interfaces frontend de base.	✓
Phase 2	Semaine 3–4	Nettoyage du dataset, développement du pipeline NLP et du modèle LSTM. Intégration du dashboard analytique.	✓
Phase 3	Semaine 5–6	Finalisation du backend FastAPI, moteur KNN, automatisation RPA (workflows Ui-Path), et intégration globale.	✓
Phase 4	Semaine 7–8	Tests unitaires, correction de bugs, sécurisation, rédaction du rapport et préparation de la soutenance.	✓